Ajman University

College of Engineering and Information Technology

Department of Electrical Engineering



COE349: Embedded Systems

Embedded Systems Project

Team Members:

Name: Mojtaba Eidyan Dalaki **ID: 202210045**

Name: Sayem Ibne Islam **ID: 202120153**

Name: Mazen Ehab **ID: 201910466**

Theory Instructor: Dr. Khalid S. Ammar
Lab Instructor: Eng. Ahmed Osman
Section: Tuesday Thursday 11:00 - 12:29

2023 - 2024

SMART VERTICAL FARM

❖ Project Schedule:

Task	Duration(weeks)			
	1	2	3	4
Planning				
Project requirement				
Analyzing and design				
Project development				
Project test				

Table (1) Project Schedule

❖ System limitation:

System limitation may be having influence on the system performance, or can affect the system implementation, we explained some of limitation that we faced throughout system implementation:

1. The cost limitation may have the biggest impact on the project implementation.

2. Availability of some important sensors used in the system in the markets, and ability to purchase these sensors.

3. Ability to find suitable alternative components and solutions for the system when not available.

4. Reduced project implementation because of lacking time to complete the full project.

❖ Introduction:

Traditional farming methods are facing increasing challenges due to limited land availability and unpredictable weather conditions.

Most farms we know spread out horizontally, but vertical farming is different. Instead of fields, it's like a stack of gardens growing upwards, indoors. It's a high-tech way to grow food in cities and places where land is scarce. By stacking plants on top of each other, we can save space and use less land. Plus, we can control everything the plants need, like light, water, and temperature, using smart technology, making it a promising solution for sustainable food production in urban areas and beyond.

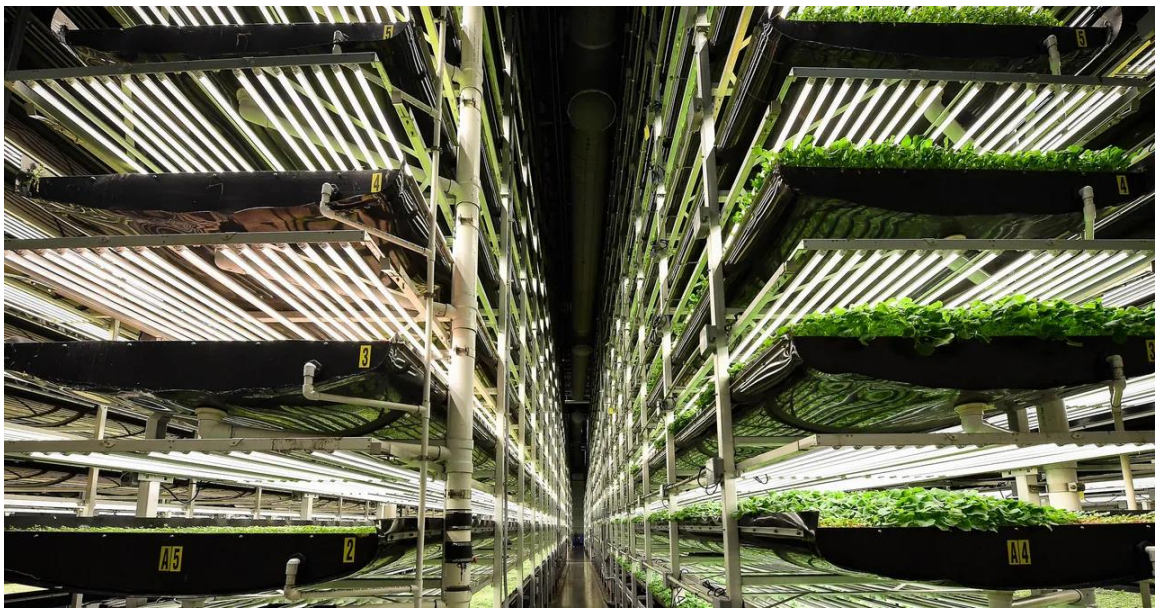


Figure (1): Vertical farming

This project is about making vertical farming even smarter by using technology to automatically manage these conditions. It's like having a high-tech gardener that takes care of everything!

❖ Theoretical Background:

Traditional farms grow crops outside, relying on sunlight, soil, and rain. But vertical farms do things differently. They grow plants indoors, in layers, without soil.

Vertical farming is a new way of growing crops indoors, kind of like stacking gardens on top of each other in a building. Instead of using soil, plants are grown in water with added nutrients, or sometimes in a special material. This helps the plants grow well because they get exactly what they need to grow big and healthy.

To make sure the plants have everything they need, we use special lights that sunlight and gadgets that keep the air just right. These gadgets measure things like temperature to make sure it's just perfect for the plants. We also use smart machines to do things like watering and harvesting, so we don't have to do it all by hand.

We use fancy gadgets to make sure the plants have enough light, water, and the right temperature to grow well. It's like having a super garden where we control everything to help the plants grow big and healthy, even in the city, this is where smart technology plays a crucial role. By using sensors and automated systems, we can create optimal growing conditions for plants, regardless of external factors.

So, in simple terms, vertical farming works by growing crops indoors in a smart way. We use special lights, gadgets, and machines to create the perfect conditions for plants to grow, without needing lots of space like traditional farms.

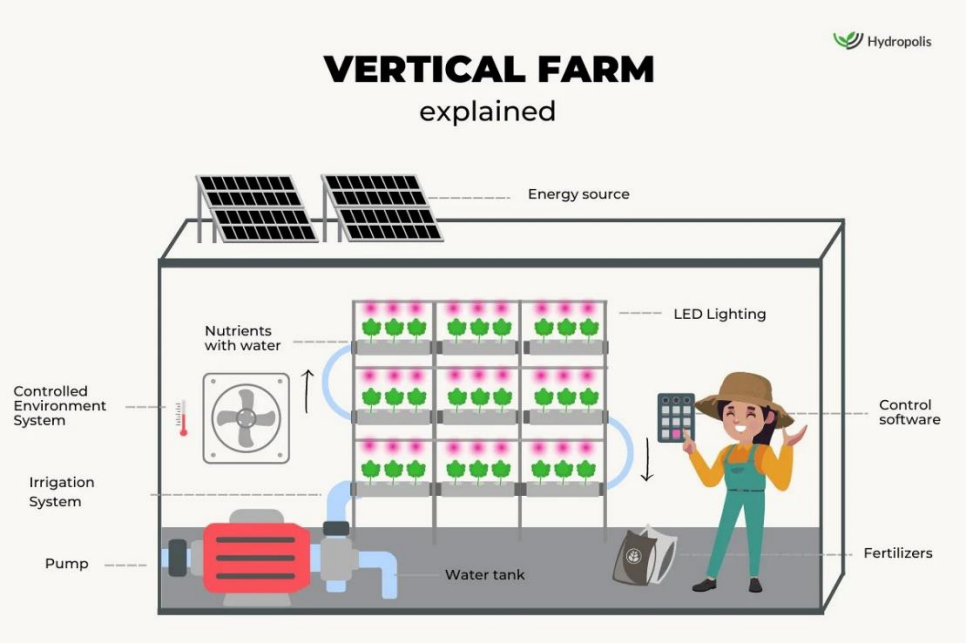


Figure (2): Vertical Farming Technology

Benefits of Vertical Farming:

Space Efficiency: Vertical farming saves space by growing crops in stacked layers, like shelves in a garden.

Continuous Harvesting: Unlike traditional farms that depend on seasons, vertical farms can produce food all year long. We don't have to worry about bad weather ruining our crops or waiting for the right time to plant.

Resource Conservation: Vertical farming uses less water and land compared to traditional farming. Also, it often recycles water and nutrients.

Fresh Produce: Vertical farms can be set up near cities, so the food travels shorter distances from farm to table. This means fresher produce and less pollution from transportation.

❖ Advantages:

Reduced Environmental Impact: Vertical farming uses fewer pesticides and herbicides because it's done indoors. This means less pollution and healthier food.

Less Dependency on Weather: Since vertical farms are indoors, they're not affected by weather conditions like storms or anything else.

❖ Disadvantages:

High Initial Costs: Setting up a vertical farm requires a lot of investment in equipment and infrastructure, which can be expensive.

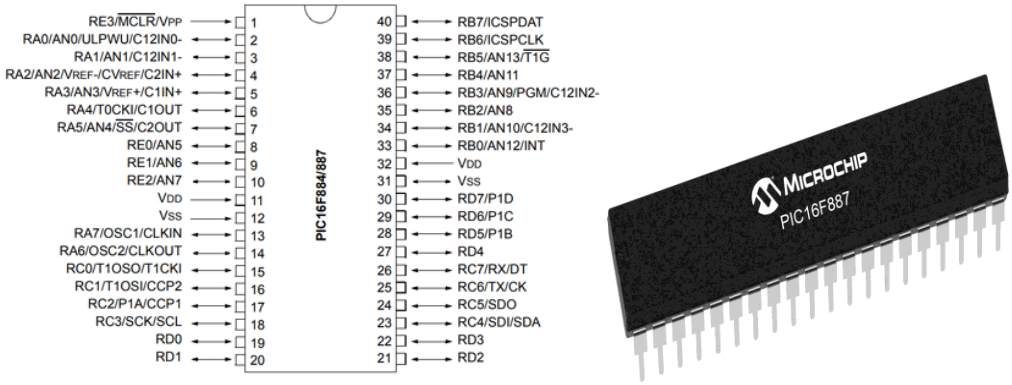
Energy Consumption: Vertical farms need electricity to power their lights and other equipment, which can lead to high energy bills.

Technical Complexity: Operating a vertical farm requires knowledge of advanced farming techniques and technology.

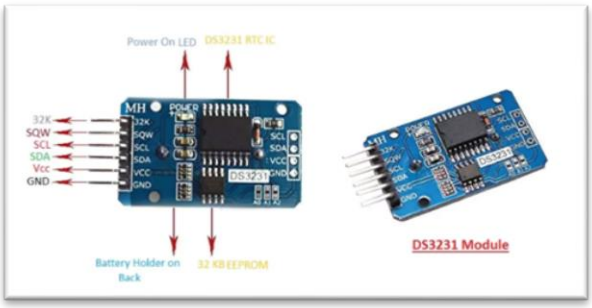
❖ Project Overview:

Our project uses various components:

- **PIC16F887 microcontroller:** This acts as the brain of the system, controlling all operations.



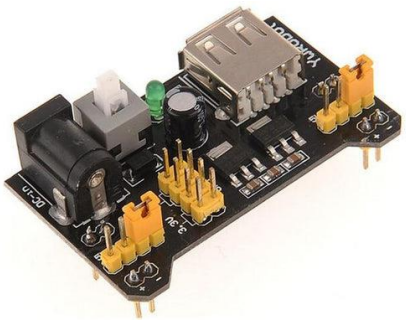
- **DS3231 real-time clock**: It keeps track of time for scheduling tasks like watering and lighting and temperature.



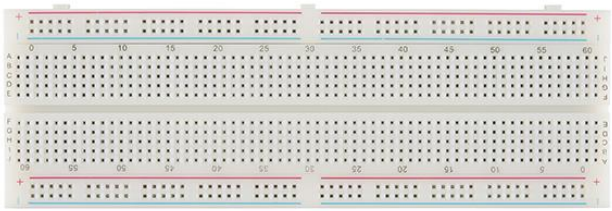
- **LCD2004 display:** This provides a user interface for displaying real time data and many other things.



- **3V Coin Cell Battery:** This battery provides backup power to the RTC module (DS3231), ensuring accurate timekeeping even in the event of a power outage.
- **Power Source with 5V:** A stable 5V power source is for powering the microcontroller, and other electronic components in the system. which we used "MB102 Breadboard Power Supply Module "it provide regulated power to a breadboard.



- **Breadboard:** The breadboard serves as a platform for prototyping and connecting various electronic components.



- **Jumper Wires:** Jumper wires are used to establish electrical connections between different components on the breadboard.
- **PIC Programmer (PICkit 2):** A PIC programmer is essential for programming the microcontroller with the required firmware. It allows us to upload the code to the microcontroller.



Project Implementation:

This code initializes communication with an LCD display and a RTC module, setting up necessary pin configurations and functions for reading and writing data with I2C communication. It defines functions to read and write data from the DS3231 real-time clock module, as well as helper functions to convert temperature values and set initial time and date. In the main function, it initializes the LCD display, sets the initial time and date, and continuously reads temperature, date and time data from the DS3231 module. Based on the temperature readings, it controls the state of certain ports to adjust environmental conditions like lighting, fan, watering and displays the current time, date, and temperature on the LCD display.

Temperature Control (Fan Activation): If the temperature reading exceeds 24°C, indicating higher temperatures, the microcontroller activates Port B, Pin 0 to trigger a fan for cooling purposes. Also, if the temperature drops below 18°C, indicating cooler conditions, PORTB.F0 is deactivated to conserve energy. This temperature regulation ensures optimal growing conditions for the crops.

Lighting Control: Based on the hour data, if it is between 6 PM and 6 AM (nighttime), PORTB.F1 is set high to indicate the need for lighting. During 6 AM to 6 PM (daytime), PORTB.F1 is set low to conserve energy.

Watering Control: The system monitors minute data and activates PORTB.F2 every five minutes. This periodic activation serves as an indicator for initiating watering mechanisms. Every five minutes (for example), PORTB.F2 is set high to trigger the watering system, ensuring regular hydration for the crops. After the watering action, PORTB.F2 is set low to indicate the end of the watering cycle.

❖ DS3231 Module Capabilities:

While the project primarily focuses on the DS3231 module for timekeeping functionality within the smart vertical farming system, it's important to note that the DS3231 offers a range of capabilities beyond those in this project. Some additional features [2] of the DS3231 include:

Alarm: The DS3231 module includes two programmable alarms, which can be configured to trigger at specific times or intervals. These alarms can be utilized for scheduling tasks or alerting users to specific events.

Day of the Week: In addition to providing accurate time and date information, the DS3231 module also tracks the day of the week. This feature adds further precision to timekeeping and enables more advanced scheduling options.

Battery Backup: With its built-in backup battery input, the DS3231 module can retain timekeeping functionality even in the absence of external power.

And many more such as Low Power Consumption, I2C Interface and …

❖ Code Explanation:

```
sbit LCD_RS at RD0_bit;
sbit LCD_EN at RD1_bit;
sbit LCD_D4 at RD2_bit;
sbit LCD_D5 at RD3_bit;
sbit LCD_D6 at RD4_bit;
sbit LCD_D7 at RD5_bit;

sbit LCD_RS_Direction at TRISD0_bit;
sbit LCD_EN_Direction at TRISD1_bit;
sbit LCD_D4_Direction at TRISD2_bit;
sbit LCD_D5_Direction at TRISD3_bit;
sbit LCD_D6_Direction at TRISD4_bit;
sbit LCD_D7_Direction at TRISD5_bit;

unsigned short ds3231_read(unsigned short address) {
   unsigned short read_data;
   I2C1_Start();
   I2C1_Wr(0xD0);
   I2C1_Wr(address);
   I2C1_Repeated_Start();
   I2C1_Wr(0xD1);
   read_data = I2C1_Rd(0);
   I2C1_Stop();
   return(read_data);
}

unsigned short ds3231_write(unsigned short address, unsigned short write_data) {
   I2C1_Start();
   I2C1_Wr(0XD0);
   I2C1_Wr(address);
   I2C1_Wr(write_data);
   I2C1_Stop();
}

unsigned char LSB(int x) {
   return((x & 0x0F) + '0');
}

unsigned short MSB(int x) {
   return((x>>4) + '0');
}

void set_initial_time_date() {
   ds3231_write(0, 0x00);
   ds3231_write(1, 0x35);
   ds3231_write(2, 0x12);
   ds3231_write(4, 0x20);
```

```
    ds3231_write(5, 0x04);
    ds3231_write(6, 0x24);
}

int second;
int minute;
int hour;
int day;
int month;
int year;
char time[] = "00:00:00";
char date[] = "00/00/2000";
unsigned short temperature_MSB, temperature_LSB;
float temperature_celsius;
char temp_buffer[10];

void main() {

    Lcd_Init();
    I2C1_Init(100000);
    I2C1_Start();
    Lcd_Cmd(_LCD_CURSOR_OFF);
    Lcd_Cmd(_LCD_CLEAR);
    Lcd_Out(1, 3, "AJMAN UNIVERSITY");
    Lcd_Out(2, 2, "SMART VERTICAL FARM");
    Lcd_Out(3, 2, "PROJECT BY MOJTABA");
    Lcd_Out(4, 3, "SAYEM AND MAZEN");
    delay_ms(3000);
    Lcd_Cmd(_LCD_CLEAR);
    Lcd_Out(1, 3, "AJMAN UNIVERSITY");
    Lcd_Out(2, 1, "Time:");
    Lcd_Out(3, 1, "Date:");
    Lcd_Out(4, 1, "Temp:");

    set_initial_time_date();
    Trisb = 0;

    while(1) {
        temperature_MSB = ds3231_read(0x11);
        temperature_LSB = ds3231_read(0x12);

        temperature_celsius = (temperature_MSB & 0x7F) + (temperature_LSB >> 6) * 0.25;
        if (temperature_MSB & 0x80) {
            temperature_celsius = -temperature_celsius;
        }

        second = ds3231_read(0);
        minute = ds3231_read(1);
        hour = ds3231_read(2);
        day = ds3231_read(4);
```

```
        month = ds3231_read(5);
        year = ds3231_read(6);


if (temperature_celsius > 24) {
        PORTB.F0 = 1;
    } else if (temperature_celsius < 18){
        PORTB.F0 = 0;
    }
      delay_ms(100);

    if (hour >= 18 || hour < 6) {
        PORTB.F1 = 1;
        } else if (hour < 18 || hour > 6) {
        PORTB.F1 = 0;
        }
        delay_ms(100);

    if ((minute  == 0 )
        PORTB.F2 = 1;
        else {
        PORTB.F2 = 0;
        }
         delay_ms(100);

        time[0] = MSB(hour);
        time[1] = LSB(hour);
        time[3] = MSB(minute);
        time[4] = LSB(minute);
        time[6] = MSB(second);
        time[7] = LSB(second);

        date[0] = MSB(day);
        date[1] = LSB(day);
        date[3] = MSB(month);
        date[4] = LSB(month);
        date[8] = MSB(year);
        date[9] = LSB(year);

        FloatToStr(temperature_celsius, temp_buffer);
        strcat(temp_buffer, " C");


        Lcd_Out(3, 1, "Temp:");
        Lcd_Out(3, 6, temp_buffer);
        Lcd_Out(1, 6, time);
        Lcd_Out(2, 6, date);
        delay_ms(500);
    }
}
```

1. LCD Configuration:

The first 12 lines of the code configure the connections between the microcontroller and the LCD display. Each LCD pin is assigned to a specific port pin of the microcontroller, and their direction is set accordingly using the sbit and sbit_direction commands. This configuration ensures that the microcontroller can communicate with the LCD display effectively.[1]

2. Real-Time Clock Functions:

The next part of the code defines two functions for reading and writing data to the DS3231 real-time clock module.

The ds3231_read function reads data from the specified address of the DS3231 module using I2C communication protocol, The address 0xD0 is the 7-bit address of the DS3231 module for write operations, The address argument passed to the function specifies the memory location to be read. Then "I2C1_Repeated_Start(); " allows the microcontroller to change from a write operation to a read operation without ending the communication sequence.
Line "I2C1_Wr(0xD1)" sends the I2C address of the DS3231 module for reading data. The address 0xD1 is the 7-bit address of the DS3231 module for read operations.
Then it reads data from the DS3231 module and assigns it to the read_data variable. The I2C1_Rd(0) function reads a single byte of data from the DS3231 module with the acknowledgment bit disabled, indicating that it's the last byte to be read, and at the end stop condition for the I2C communication protocol.
while the ds3231_write function writes data to the specified address of the module. These functions enable the microcontroller to interact with the DS3231 module to retrieve and set time and date information.[2]

3. Helper Functions:

Following the real-time clock functions, there are two helper functions: LSB and MSB. These functions are used to extract the least significant and most significant nibbles which are 4 bits of an integer, respectively. They are used for formatting time and date values for display on the LCD.

4. Initial Time and Date Setup:

The set_initial_time_date function initializes the DS3231 module with default time and date values. It writes specific values to the appropriate addresses of the module to set the initial time and date. This function is called at the beginning of the main function to ensure that the system starts with accurate timekeeping.
Once microcontroller is programmed by code, this initial case shall be removed otherwise after turning off the system it will be reset to initial values.

5. Main Function:

The main function is the entry point of the program. It initializes the LCD display, sets up communication with the DS3231 module, clears the LCD screen, and displays initial messages. Then, it calls the set_initial_time_date function to set the initial time and date. Inside the main loop, it continuously reads temperature and time data from the DS3231 module, adjusts environmental conditions based on temperature readings, updates the time and date display on the LCD, and delays for a short period before repeating the process. This loop ensures that the system continuously monitors and displays relevant information on the LCD screen.

ADDRESS	BIT 7 MSB	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0 LSB	FUNCTION	RANGE
00h	0	10 Seconds			Seconds				Seconds	00–59
01h	0	10 Minutes			Minutes				Minutes	00–59
02h	0	12/$\overline{24}$	$\overline{AM}$/PM 20 Hour	10 Hour	Hour				Hours	1–12 + $\overline{AM}$/PM 00–23
03h	0	0	0	0	0	Day			Day	1–7
04h	0	0	10 Date		Date				Date	01–31
05h	Century	0	0	10 Month	Month				Month/ Century	01–12 + Century
06h	10 Year				Year				Year	00–99

11h	SIGN	DATA	DATA	DATA	DATA	DATA	DATA	DATA	MSB of Temp	—
12h	DATA	DATA	0	0	0	0	0	0	LSB of Temp	—

Figure (3): Ds3231 Reading According to Datasheet

❖ Circuit Diagram:

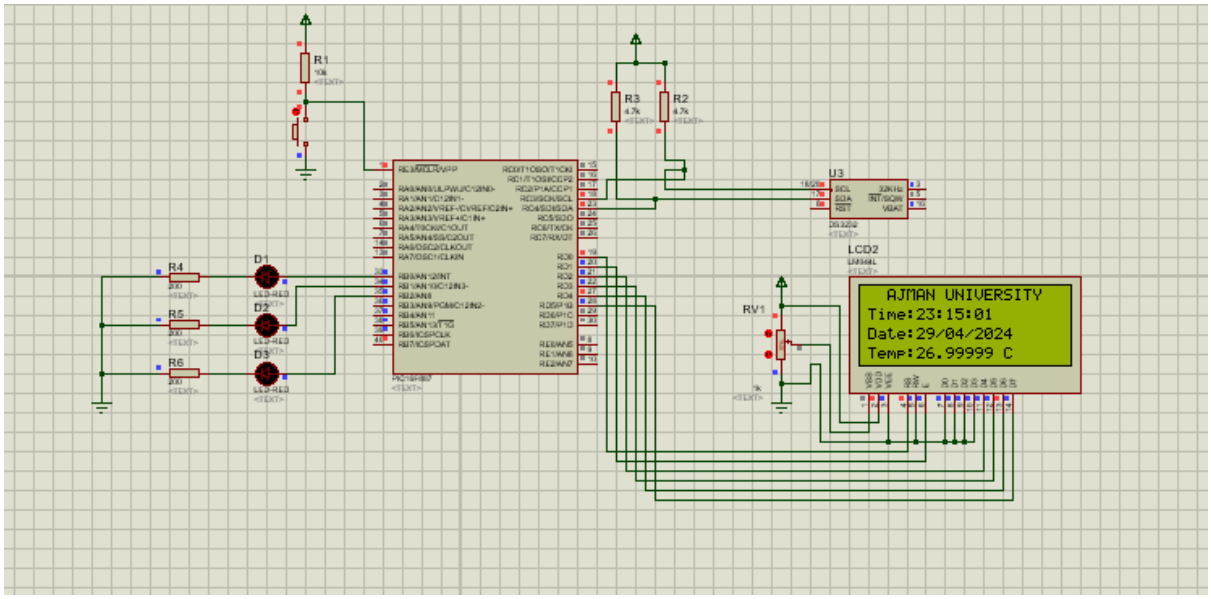


Figure (4): Circuit Diagram in Proteus

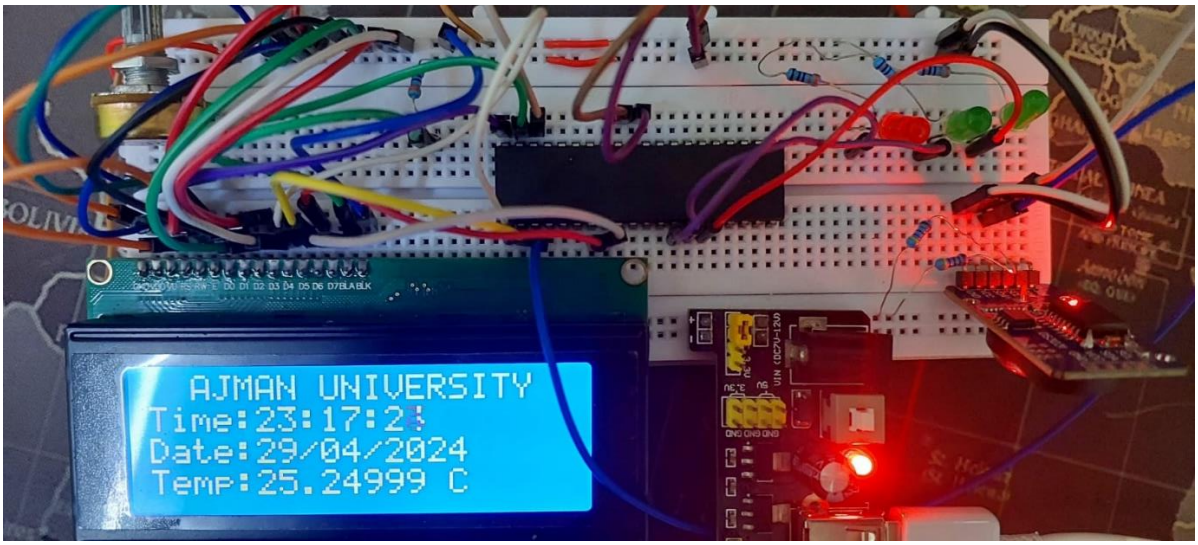


Figure (5): Circuit Implementation

❖ Pickit2 Programming:

The PICkit 2 Development Programmer/Debugger is a low-cost development programmer. It is capable of programming most of Microchipís Flash microcontrollers and serial EEPROM devices. For specific device support, refer to the datasheet of pickit2.

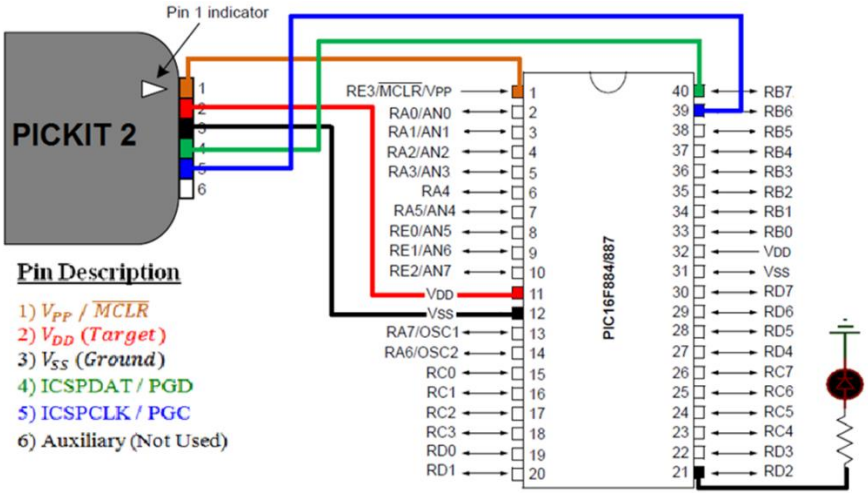


Figure (6): Connection PICKIT2 Programmer to PIC16F887

PICkit 2 Programmer Application:

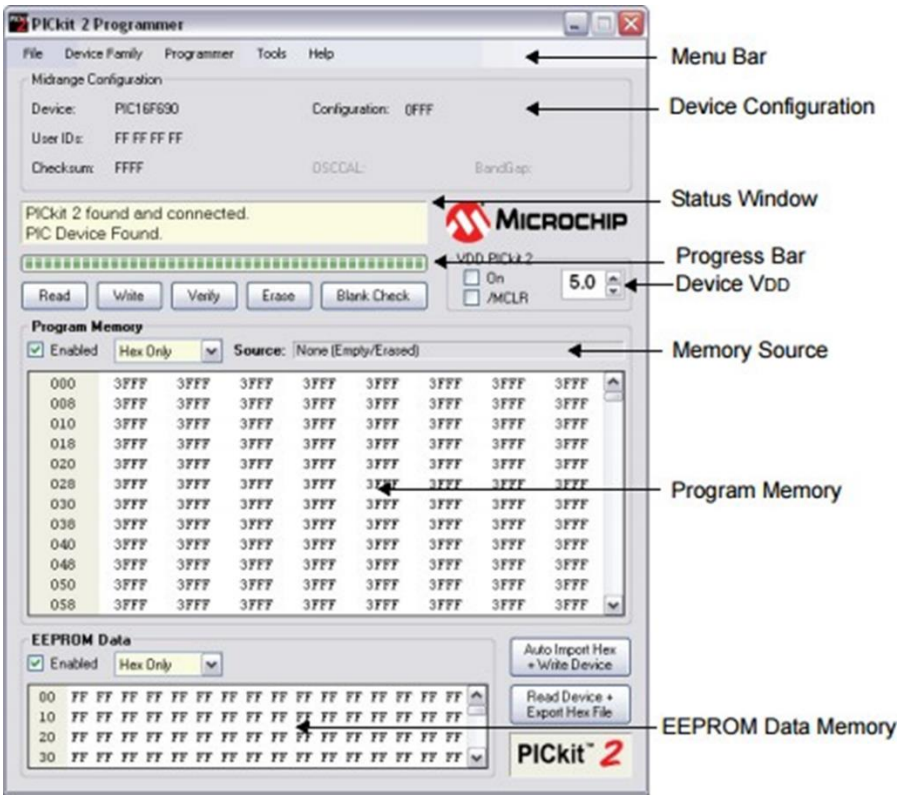
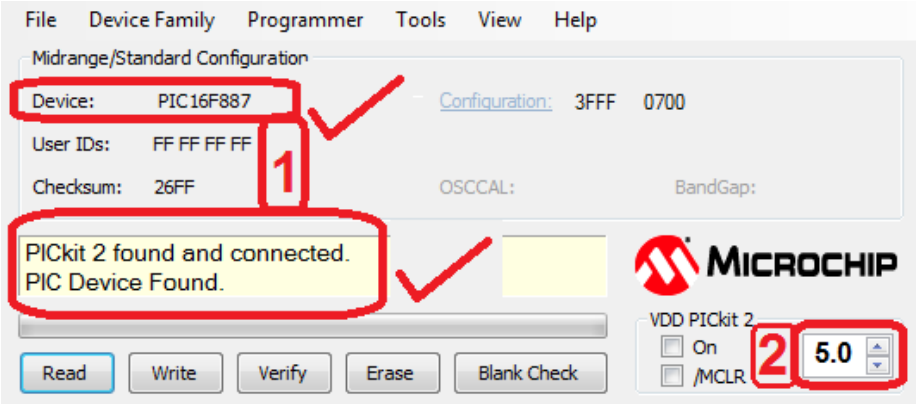
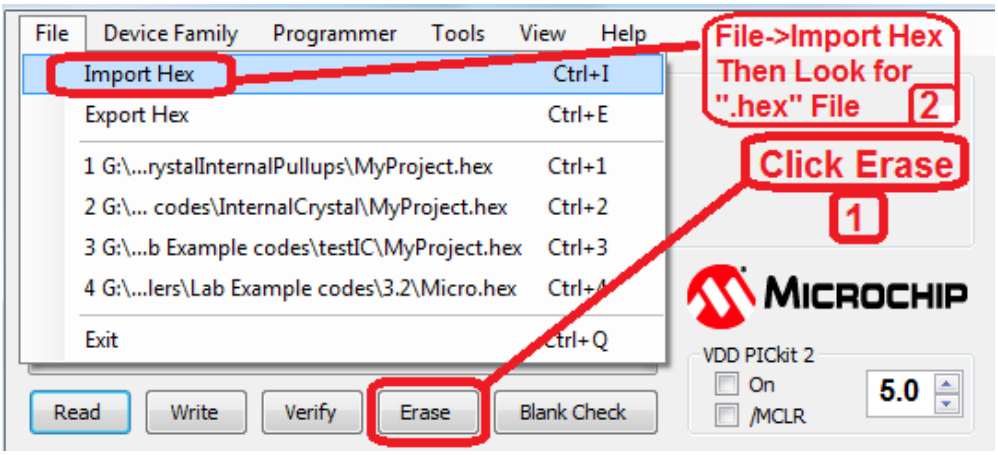


Figure (7): Configuration of PICKIT2 Programmer

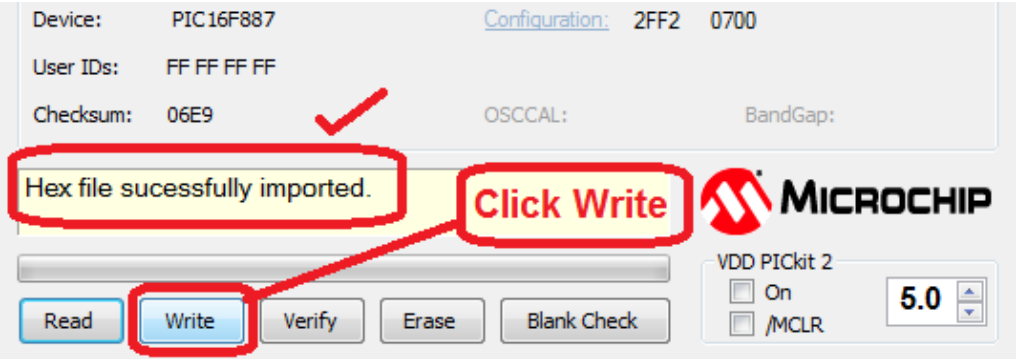
Steps:



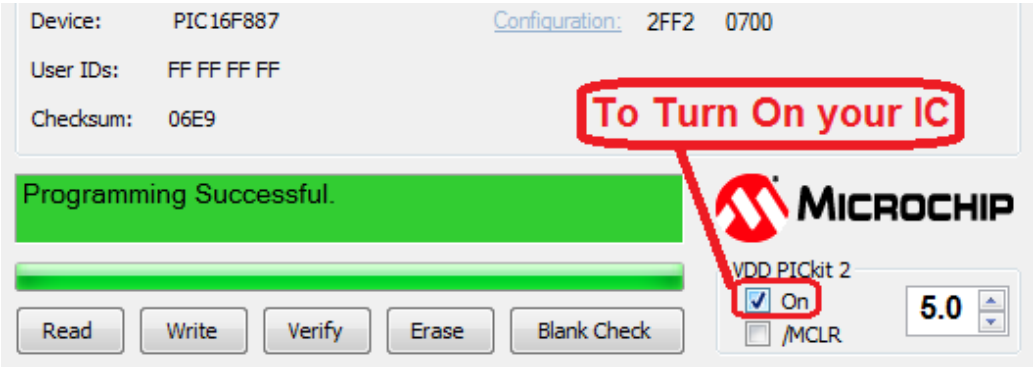
Step #1: Initializing PICKIT 2 Program



Step #2: Importing Hex File to PIC16F887



Step#3: Writing Hex File on PIC16F887



Step#4: Turning ON PIC16F887 using PICKIT 2 APP

After programming with the PICkit2, we can disconnect it from the circuit. Then, we'll need to power the circuit using a suitable power source, such as a battery pack or a DC power adapter. In this case, the MB102 Breadboard Power Supply Module can be connected to a power bank via a USB cable to provide the necessary regulated voltages (typically 3.3V and 5V) to the circuit components.

❖ Cost table:

Piece	Price (DHS)
Real time clock module Ds3231	20
CR2032 3v battery	5
2004 LCD	30
10k Ohm potentiometer	5
4.7K and 330 Ohm resistors	10
MB102 Breadboard Power Supply	25
Breadboard	10
Jumper wires (40PCS)	10
Total	**115**

Table (2) Costs

❖ Conclusion:

In short, smart vertical farming uses technology to help grow crops better in cities where space is tight. By controlling things like fan, light and water automatically, it helps crops grow well and saves resources. In the future, this type of farming could change how we grow food, making it easier and more efficient for everyone.

❖ References:

[1] Microchip Technology Inc. (2012). PIC16F887 Datasheet. Retrieved from [https://ww1.microchip.com/downloads/en/devicedoc/41291d.pdf]

[2] Analog Devices. (2015). DS3231 Datasheet. Retrieved from [https://www.analog.com/media/en/technical-documentation/data-sheets/DS3231.pdf]